WALZ 4.0

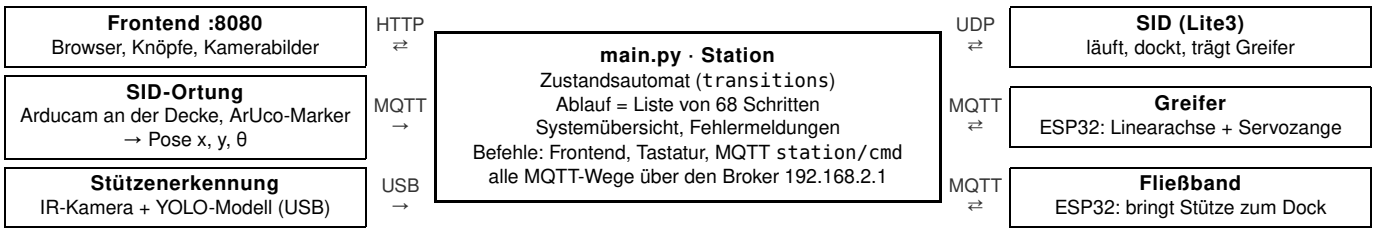
STATION – FUNKTION DES CODES

Stützen verteilen und einsammeln · Python, MQTT, Browser-Frontends

Stand 25.10.2026
Ausführlich: docs/DOKUMENTATION.md

Ein vierbeiniger Roboter (**SID**, DEEP Robotics Lite3) mit Greifer auf dem Rücken verteilt drei Metallstützen auf feste Plätze und holt sie wieder ab. Ein Linux-PC steuert alles: `main.py` führt den Ablauf aus (Frontend :8080), `trainer.py` bringt dem SID neue Bewegungen bei (eigenes Programm, Frontend :8081).

AUFBAU



ABLAUF „START“

1. **Ausliefern ×3:** YOLO erkennt eine Stütze → ein Mensch stellt sie aufs Band (10 s) → Band fährt 18 s zum Dock. Gleichzeitig fährt der SID zur Dock-Vorposition, dockt, greift, fährt heraus und stellt die Stütze auf ihren Platz (2,0 | –0,6 / 0 / 0,6).
2. **Pause:** 30 s am Warteplatz (1,5 | 0).
3. **Zurückholen ×3:** anfahren, greifen, zur Abgabe (1,0 | 0,25) tragen, rückwärts heraus.

`liefern` = nur 1, `holen` = nur 3. Pause hält nach dem laufenden Schritt an.

ZUSTANDSAUTOMAT

```
init → idle ⇄ running ⇄ paused
running → idle | docked | error
* → estop · error/estop → reset → idle
```

Arbeits-Threads (Ablauf, Band, Navigation) melden ihr Ergebnis als Trigger zurück; der Automat ist threadsicher. Fehlt beim Start die Pose, misst die Station automatisch neu, sobald die Ortung den SID sieht.

NAVIGATION

Gelernte **Laufmodelle** (Physik-Grundlinie + Gaußprozess) sagen voraus, wohin ein Befehl (vor, seitlich, drehen …, Dauer) den SID bringt. Regelkreis pro Schritt:

`messen → planen → ausführen → stoppen → messen`

Grob: drehen + gerade laufen; fein: Suche über Befehlsfolgen. Ziel ±5 cm / 5°. Docken, Ausdocken und Abgabe laufen gerade ohne Drehen.

SICHERHEIT

- Sicherheitszone x < 1,2 m: nur Dock, Abgabe, Ausdocken – gerade, ohne Drehen.
- Stopp, wenn die Pose älter als 1,5 s ist oder der SID > 4 m vom Start entfernt.
- **NOT-AUS** (Leertaste, `estop`): SID, Band, Achse sofort stoppen.
- Training fährt nur, wenn die Station aus ist und die Ortung trackt.

BEDIENUNG

Station (Symbol „Station“): `F2` Start · `F3` Pause/Weiter · `F4` Stop · `F8` Quittieren · `Leertaste` **NOT-AUS** · `F10` Beenden. Rechts oben „Systeme“: Broker, Ortung, SID, Band, Greifer, YOLO in grün/gelb/rot.

Bewegungstraining (Symbol „Bewegungstraining“): Policy wählen (gerade, gemischt, Drehungen, fein) → Daten sammeln (SID läuft Zufallsbefehle im 1-m-Umkreis) → trainieren → Kandidat übernehmen.

CODE-LANDKARTE

main.py	Zustandsautomat, Ablauf, Befehle, Start und Ende
navigate.py	Planer, Sicherheitszone, Docken · `lite3_commander.py`: UDP an SID
aruco_tracker_calibrated.py	Ortung: Kamerabild → Pose per MQTT
arm_controller.py	Greifer (Grenzen in `arm_config.py`)
conveyor_controller.py	Fließband
F_stilt_erkennung_YOLO.py	IR-Kamera + YOLO
station_frontend.py	Frontend der Station, Systemübersicht
trainer_server.py	Bewegungstraining (mit `trainer.py`)
collect_*/train_*.py	Daten sammeln / Modelle trainieren

SCHNITTSTELLEN UND DATEN

station/cmd	Befehle als Text (start, stop, estop …)
station/state	Zustand als JSON (retained)
station/tracker/…	Pose, Status, Vorschaubild der Ortung
station/arm/…, conveyor…	Greifer und Band: Befehle, Zustand

`models/` aktive Modelle · `data/` Trainingsdaten, Fahrlogs, Diagramme · `config/` Frontend und Tasten · `logs/` eine Logdatei pro Start.

Start: Desktop-Symbol oder `python main.py` / `python trainer.py`

Mehr: KURZFASSUNG.md · DOKUMENTATION.md · INBETRIEBNAHME.md